CLRS 4판 기반 공부용 한국어 요약

그래프부터 위상정렬까지

이 문서는 *Introduction to Algorithms, 4th Edition*의 그래프 기초부터 위상정렬까지를 재혁님 공부용으로 쉽게 풀어쓴 요약본입니다. 원문을 그대로 옮긴 번역이 아니라, 핵심 개념을 이해하기 좋게 정리한 학습용 번역·요약입니다.

1. 그래프란 무엇인가

그래프는 **정점(vertex)**과 **간선(edge)**으로 이루어진 자료구조입니다. 정점은 어떤 대상, 간선은 두 대상 사이의 관계를 뜻합니다.

- 정점: 사람, 도시, 작업, 웹페이지처럼 하나의 대상
- 간선: 친구 관계, 도로 연결, 작업 순서, 링크처럼 두 대상을 잇는 관계
- 방향 그래프: 간선에 방향이 있음. 예: `A → B`
- 무방향 그래프: 간선에 방향이 없음. 예: `A - B`

> 그래프는 "무언가가 서로 연결되어 있다"는 상황을 표현하기에 아주 좋은 도구입니다.

2. 그래프를 저장하는 방법

2-1. 인접 리스트

각 정점마다 "나와 연결된 정점들"의 목록을 저장하는 방식입니다. 간선 수가 많지 않을 때 메모리를 아낄 수 있어서 실전에서 자주 씁니다.

- 장점: 메모리를 적게 쓰는 편이고, 연결된 이웃을 바로 볼 수 있음
- 단점: 두 정점이 바로 연결되었는지 확인할 때는 목록을 찾아봐야 할 수 있음

2-2. 인접 행렬

정점 수가 `V` 개일 때 `V × V` 표를 만들고, 두 정점이 연결되었는지 칸으로 표시하는 방식입니다.

- 장점: 연결 여부를 아주 빨리 확인 가능

- 단점: 간선이 적어도 큰 표를 써야 해서 메모리 사용량이 큼

언제 인접 리스트가 좋은가?

간선 수가 적거나, 탐색을 많이 할 때 좋습니다. 알고리즘 문제에서도 자주 등장합니다.

언제 인접 행렬이 좋은가?

정점 수가 작고, 특정 두 점의 연결 여부를 자주 즉시 확인해야 할 때 편합니다.

3. 너비 우선 탐색 BFS

BFS(Breadth-First Search)는 시작점에서 가까운 정점부터 차례대로 넓게 퍼져 나가며 탐색하는 방법입니다. 보통 **큐(queue)**를 사용합니다.

핵심 아이디어

- 시작 정점을 먼저 방문한다.
- 시작 정점과 직접 연결된 정점들을 본다.
- 그다음 거리가 2인 정점들, 3인 정점들 순서로 본다.

왜 중요한가

- 가중치가 없는 그래프에서 최단 거리를 구할 수 있습니다.
- 정점이 어떤 "레벨"에 있는지 파악할 수 있습니다.

BFS가 자연스럽게 맞는 상황

미로에서 최소 이동 횟수 찾기, 사람 사이의 촌수 계산, 한 번에 퍼지는 전염/소식 문제

시간복잡도

인접 리스트로 구현하면 보통 `O(V + E)`입니다. 각 정점과 간선을 많아야 한 번씩만 의미 있게 보기 때문입니다.

4. 깊이 우선 탐색 DFS

DFS(Depth-First Search)는 한 방향으로 갈 수 있을 때까지 깊게 들어간 뒤, 더 갈 곳이 없으면 되돌아와 다른 길을 탐색하는 방식입니다. 보통 **재귀** 또는 **스택**으로 구현합니다.

핵심 아이디어

- 시작점에서 출발한다.
- 방문하지 않은 이웃이 있으면 그쪽으로 계속 내려간다.
- 막히면 돌아와서 다른 이웃을 본다.

DFS에서 자주 보는 개념

- 발견 시간: 어떤 정점을 처음 만난 순간
- 종료 시간: 그 정점에서 갈 수 있는 탐색을 모두 끝낸 순간
- 부모 정점: DFS 트리에서 나를 처음 방문하게 만든 정점

CLRS에서는 DFS를 단순 방문 도구가 아니라, 그래프 구조를 분석하는 강력한 기반 도구로 다룹니다. 연결 요소, 사이클, 위상정렬, 강한 연결 요소 같은 주제들이 DFS와 이어집니다.

시간복잡도

DFS도 인접 리스트 기준으로 보통 `O(V + E)` 입니다.

5. BFS와 DFS 차이

BFS

- 가까운 곳부터 넓게 탐색
- 큐 사용
- 무가중치 최단 거리 문제에 강함

DFS

- 한 길을 끝까지 깊게 탐색
- 재귀나 스택 사용
- 사이클, 위상정렬, 연결 구조 분석에 강함

6. 위상정렬 Topological Sort

위상정렬은 **방향 비순환 그래프(DAG)**에서 정점들을 나열하는 방법입니다. 모든 간선 `u → v`에 대해, 순서에서 반드시 `u`가 `v`보다 앞에 와야 합니다.

쉽게 말하면

"선행 작업을 먼저 하고, 그 다음 작업을 한다"는 순서를 만드는 것입니다. 예를 들어, 알고리즘 수업을 듣기 전에 자료구조를 먼저 배워야 한다면 자료구조가 앞에 와야 합니다.

왜 DAG에서만 가능한가

사이클이 있으면 서로가 서로보다 먼저 와야 하는 모순이 생깁니다. 예를 들어 `A → B → C → A`라면, A가 B보다 앞, B가 C보다 앞, C가 A보다 앞이어야 하므로 순서를 만들 수 없습니다.

DFS와 위상정렬의 연결

CLRS에서는 DFS를 이용해 각 정점의 **종료 시간**을 구하고, 종료 시간이 큰 순서대로 정점을 나열하면 위상정렬이 된다고 설명합니다.

- 깊게 들어가며 탐색한다.
- 탐색이 끝난 정점을 스택에 넣거나 앞쪽에 배치한다.
- 모든 정점을 처리한 뒤 역순으로 읽으면 위상정렬 순서가 된다.

위상정렬이 쓰이는 예시

과목 선수 관계, 빌드 순서, 프로젝트 작업 순서, 의존성 설치 순서

7. 공부할 때 꼭 잡아야 하는 포인트

1. 그래프 표현 두 가지를 구분할 수 있어야 합니다.
2. BFS는 "거리"와 "레벨"에 강하다는 점을 기억하면 좋습니다.
3. DFS는 "구조 분석"에 강하다는 점을 기억하면 좋습니다.
4. 위상정렬은 DAG에서만 가능하다는 점이 핵심입니다.
5. 사이클이 생기면 위상정렬이 불가능하다는 이유를 말할 수 있어야 합니다.

8. 초보자용 한 줄 감각

- 그래프: 연결 관계를 표현하는 도구
- 인접 리스트: 연결된 곳만 저장
- 인접 행렬: 연결 여부를 표로 저장
- BFS: 가까운 곳부터 찾기
- DFS: 한 길 끝까지 파보기
- 위상정렬: 먼저 해야 할 일을 먼저 두는 순서 만들기

9. 마무리

그래프 단원은 처음 보면 용어가 많아 보이지만, 실제 핵심은 "어떻게 연결을 저장하고, 어떤 순서로 탐색하느냐"입니다. BFS와 DFS를 분명히 구분하고, DFS 종료 시간과 위상정렬의 연결만 잡아도 그래프 기초의 큰 줄기를 이해한 것입니다.

> 공부 팁: 재혁님은 작은 예시 그래프를 직접 그린 뒤, BFS와 DFS 방문 순서를 손으로 써보면 훨씬 빨리 익숙해질 수 있습니다.